

Travail de Bachelor

Auto-battler multijoueur

Étudiant	Nicolas Duprat
Superviseure	Prof. Donini Pier
Département	Technologies de l'information et de la communication (TIC)
Filière	Informatique et systèmes de communication (ISC)
Orientation	Informatique Logicielle (ISCL)
Travail proposé par	Nicolas Duprat
Année académique	2026-27

Yverdon-les-Bains, le 20.05.2026

Authentification

Par la présente, j'atteste avoir réalisé ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Nicolas Duprat

Yverdon-les-Bains, le 20.05.2026

Préambule

Ce travail de Bachelor (ci-après TB) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'Ecole.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD

Le Chef de département TIC

Yverdon-les-Bains, le 20.05.2026

Cahier des charges

Description du projet

Le projet a pour but de créer un jeu ayant un style similaire à des gros jeux déjà existants mais en y ajoutant un twist pour le rendre un peu unique. L'objectif n'est en aucun cas de créer quelque chose de révolutionnaire, mais de créer un jeu fonctionnel avec lequel les joueurs s'amuseront avec des personnages de la pop-culture. On cherche donc à réaliser un jeu jouable en ligne multijoueur auto-battler, où 8 personnes pourront s'affronter dans la même partie.

Technologie utilisés

Pour ce projet je vais utiliser le framework Play pour le développer, il aura donc pour objectif d'être jouable sur navigateur internet. Pour le stockage des données je vais utiliser le système de gestion de base de données PostgreSQL.

Valeur d'ingénierie du projet

Tout l'intérêt du projet réside dans la réalisation d'un jeu multijoueur entièrement en ligne, accessible directement depuis un navigateur web et ne nécessitant aucune installation côté utilisateur.

L'aspect multijoueur introduit des défis techniques importants, notamment la synchronisation d'état entre plusieurs clients, la gestion de la latence réseau, la cohérence des actions concurrentes et la fiabilité des communications. Le serveur doit également orchestrer plusieurs parties simultanées tout en garantissant l'intégrité des données et la continuité de l'expérience de jeu. Il faudra donc que le design de la Base de donnée permet une résistance à la panne.

La conception de la base de donnée devra aussi permettre un système de replay de partie afin que l'on puisse rejouer entièrement des parties qui auront été enregistrées sur la base de donnée.

Fonctionnalités attendues

Dans cette section, je décris les fonctionnalités qui seront attendues à la fin du projet. On listera aussi de potentielles fonctionnalités futures.

Must have

- Un système de login, où chaque utilisateur pourra se connecter et regarder ses statistiques, soit le nombre de victoires dans un premier temps
- Un système de connexion multijoueur, où l'on pourra connecter 8 personnes à la même partie afin qu'ils s'affrontent
- Un système de stockage des données utilisateurs
- Un système de replay de partie
- Un système de partie opérationnel : donc où un joueur pourrait lancer une partie pour jouer contre d'autres joueurs ou des bots. Donc cela inclut :
 - Un système de boutique pour acheter les unités
 - Un système de combat qui fera s'affronter les unités de deux joueurs à intervalles réguliers
 - Un système de gestion de son équipe afin que l'on puisse changer soit la formation de son équipe ou bien les unités constituant mon équipe.

Should have

- Un système de classe et de famille pour les unités : au-delà d'avoir une unité, en avoir plusieurs différentes de la même classe ou famille devrait accorder un bonus (cf. GDD).
- Un roster d'unités suffisamment grand : même si cela est assez subjectif, on va partir sur une base de 40 unités différentes qui seraient jouables
- Une interaction spécifique lors de l'utilisation de chaque famille : s'il y a suffisamment d'unités différentes d'une même famille dans votre équipe, cela devrait apporter une méca-

nique nouvelle lors des combats. Et le bonus ou la mécanique de classe devrait apporter un clin d'œil sympa aux mécaniques originel du jeu.

Could have

- Un système d'augment pendant la partie : durant la partie plusieurs choix aléatoires sont proposés aux joueurs afin d'améliorer leur équipe ou de changer leur partie actuelle
- Un pouvoir spécifique aux joueurs, et où chaque joueur pourrait choisir entre certains pouvoirs afin de changer la dynamique de sa partie
- Une IA complexe et challengeante, qui pourrait poser un vrai danger à des joueurs connaissant le jeu ou les jeux de ce genre

Won't have

- Une vraie direction artistique
- Une bande son

Durée du projet

Je dois mettre un travail équivalent à 13 h de travail par semaine le long du semestre, et 45 h lorsque mon semestre sera fini, donc en juillet. Cela fera un total de 450 h de travail disponibles.

Livrables

- remise du cahier des charges pour le 27.02.2026
- remise du rapport intermédiaire le 20.05.2026 avant 15 h
- remise du travail ainsi que le rapport des sources, une affiche et un résumé publiable pour GAPS le 23.07.2026 avant 11 h.

Jalon

Milestone 1 : Planification et setup du projet, finir le cahier des charges, le planning

Milestone 2 : analyse des besoins ainsi que la spécification de ces derniers

Milestone 3 : conception, réalisation de l'architecture design(stack, et UML de classe), design de database ainsi que le design de l'UI

Milestone 4 : implémentation des must have

Milestone 5 : implémentation des should have

Milestone 6 : réalisation des documents à rendre (affiche et résumé publiable)

Table des matières

Authentification	2
Préambule	3
Cahier des charges	4
Description du projet	4
Technologie utilisés	4
Valeur d'ingénierie du projet	4
Fonctionnalités attendues	5
Durée du projet	6
Livrables	6
Jalon	6
1 Introduction	10
1.1 Contexte motivation	10
1.2 Description du sujet	10
1.3 Description du jeu	11
1.4 Architecture du projet	11
2 Planification	13
2.1 Planification initiale	13
3 State of the art	14
3.1 Introduction	14
3.2 Frontend	14
3.3 Authentification	16
3.4 Connection Client-Serveur	16
3.5 Lien entre DB et Backend	18
3.6 CI/CD	19

4 Architecture Logiciel	20
4.1 Frontend	20
4.2 Backend	21
4.3 Base de données	22
4.4 CI/CD Pipeline	23
4.5 Schéma de l'architecture	25
5 Conception	26
5.1 Conception de la base de données	26
6 Conclusion	36
Bibliographie	37
Annexes	41
I Outils utilisés	42

1 Introduction

1.1 Contexte motivation

Ce projet s'inscrit dans une optique de réaliser un travail passionnant et en accord avec la direction vers laquelle j'aimerais que ma carrière se tourne.

En effet, je désire m'orienter vers le domaine du jeu vidéo à l'issue de mes études. Ce projet de bachelor est une opportunité de concevoir un jeu de manière structurée et professionnelle, afin de pouvoir utiliser ce projet comme une vitrine de mes compétences.

1.2 Description du sujet

Ce projet de bachelor consiste en la conception et le développement d'un jeu multijoueur accessible via le web. L'objectif est de créer une application interactive permettant à plusieurs joueurs de s'affronter au cours de parties, en mettant en œuvre des mécaniques de stratégie et de gestion d'équipe.

Le projet repose sur le développement d'une application complète intégrant à la fois une interface utilisateur, une logique métier et un système de persistance des données. Il s'inscrit dans une démarche de conception logicielle moderne, en utilisant une architecture distribuée permettant de séparer les différentes responsabilités du système.

Au-delà de l'aspect ludique, ce projet vise à mobiliser un ensemble de compétences acquises lors du cursus, telles que le développement web, la gestion d'une architecture multi-tiers, la communication entre client et serveur, ainsi que la mise en place de bonnes pratiques de

développement. Une attention particulière est également portée à la qualité du code, à la maintenabilité de l'application et à l'automatisation des tests et du déploiement.

1.3 Description du jeu

Le jeu consistera à jouer des parties. Chaque partie fera affronter 8 joueurs les uns contre les autres, au fur et à mesure des rounds.

Chaque joueur aura une équipe qu'il pourra composer des unités de son choix, ainsi que des points de vie. A chaque round chaque joueur recevra de l'argent et pourra alors acheter des unités et réagencer son équipe afin d'être le plus fort possible.

Un round dure un temps imparti et à la fin de ce dernier, chaque joueur devra affronter l'équipe d'un autre joueur et celui qui gagne inflige des dégâts au perdant.

Enfin, le dernier joueur avec encore des points de vie sera celui qui gagne la partie. Ce système s'inspire du genre des auto-battlers, où la stratégie repose principalement sur la composition et l'optimisation d'équipe plutôt que sur des actions en temps réel.

1.4 Architecture du projet

L'objectif est de mettre à profit toutes les connaissances que j'ai pu acquérir lors de mon bachelor dans la réalisation d'un jeu c'est pourquoi avant de développer un jeu web, je développe une application qui pourra être utiliser sur le web, et tout cela implique certaines choses que je vais mettre en place.

Mon application est donc multi-tiers et a un frontend, un backend et une base de données.

Le frontend ne contiendra pas de logique métier est aura pour seul est unique but d'afficher les différents états de l'application, soit l'écran pour se connecter en tant qu'utilisateur, le lobby une fois connecter et la partie en cours.

Le backend lui contiendra toute la logique métier du jeu, il aura pour but de simuler chaque combat et d'envoyer par la suite la simulation réaliser afin que le frontend puisse afficher au client le combat. Le backend aura aussi la responsabilité de checker les entrées utilisateur et de gérer les connections avec ce dernier.

La base de données va contenir toutes les informations des clients relative au jeu (historique des partie, etc...)

Dans une optique de bonnes pratiques de développement, un pipeline d'intégration et de déploiement continu (CI/CD) sera également mise en place afin d'automatiser les phases de test et de déploiement de l'application.

2 Planification

2.1 Planification initiale

Pour la planification initiale, l'objectif était de travailler en cascade tout du long. Cependant, ce plan a changer car je vais maintenant travailler en cascade jusqu'à avoir une version minimal qui est jouable et je passerais ensuite par itération pour implémenter toutes les classes et leur mécaniques.

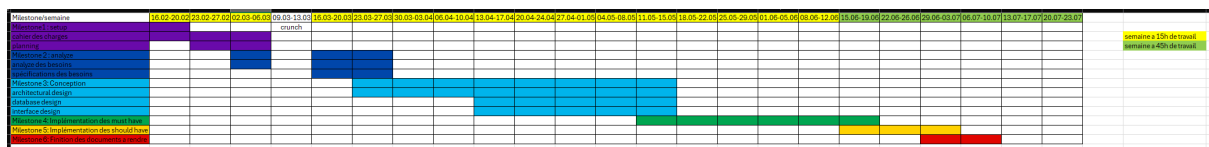


Fig. 1. – Plannification

3 State of the art

3.1 Introduction

Dans le cadre de ce projet, dont l'objectif est de développer un jeu selon des standards professionnels, ce travail s'intéresse à la manière dont différents acteurs de l'industrie répondent aux enjeux liés à ce type de développement.

Le projet repose notamment sur l'utilisation du Framework Play [1], qui propose déjà un ensemble de solutions intégrées. L'objectif est donc d'analyser les choix technologiques de solutions existante et les fonctionnalités offertes par ce Framework, afin de concevoir une infrastructure cohérente et réfléchie.

Cette étude portera plus particulièrement sur les aspects suivants : le frontend, l'authentification des utilisateurs, les communications client-serveur, l'intégration entre la base de données et le backend, ainsi que la mise en place d'un pipeline CI/CD.

3.2 Frontend

Ce projet nécessite le développement d'un frontend ne contenant aucune logique métier. Son rôle principal est d'afficher un grand nombre de sprites en deux dimensions, de permettre l'interaction avec l'utilisateur, et de transmettre des messages au backend. En effet, l'ensemble des combats est simulé et calculé côté serveur, qui constitue l'unique source de vérité. Le backend transmet ensuite les résultats de la simulation au frontend, chargé de leur affichage.

Le Framework Play propose des solutions telles que les vues Scala et les templates Twirl [2] pour le rendu de page html. Toutefois, en raison du niveau d'interactivité requis, cette approche ne semble pas adaptée. Une solution basée sur JavaScript côté client apparaît donc plus pertinente.

Dans l'écosystème JavaScript, plusieurs approches sont envisageables. Il est possible d'utiliser du JavaScript pur, ou bien des technologies bas niveau telles que WebGL [3] pour gérer le rendu graphique. Des solutions plus haut niveau existent également, comme les bibliothèques de rendu (par exemple PixiJS [4]) ou des moteurs de jeu complets tels que Phaser [5].

L'utilisation de JavaScript pur ou de WebGL offre une grande flexibilité, mais implique un coût de développement important [6]. Compte tenu des contraintes de temps du projet, ces solutions ne sont pas privilégiées, bien que WebGL puisse constituer une option intéressante dans une perspective d'évolution vers la 3D.

L'un des critères de choix concerne justement la compatibilité avec des rendus 3D. Certaines technologies, notamment les bibliothèques de rendu 2D, ne supportent pas cette évolution, ce qui pourrait nécessiter une refonte importante en cas de changement futur.

Le choix se restreint ainsi entre une bibliothèque de rendu et un moteur de jeu. La principale différence réside dans les fonctionnalités fournies : un moteur de jeu intègre des éléments complets tels que la boucle de jeu, la gestion du son ou des interactions, tandis qu'une bibliothèque de rendu se concentre principalement sur l'affichage graphique.

Dans le cadre de ce projet, une grande partie des fonctionnalités offertes par un moteur de jeu ne sont pas utiles, notamment la gestion des collisions, puisque la logique métier est entièrement gérée côté backend.

Selon des benchmarks existants [7], [8], PixiJS apparaît comme l'une des solutions les plus performantes pour l'affichage d'un grand nombre de sprites avec un taux de rafraîchissement élevé.

Le choix de PixiJS permet ainsi d'optimiser les performances d'affichage. Toutefois, cette approche nécessite l'ajout de fonctionnalités complémentaires, telles que la gestion du son ou de certaines interactions. Ces besoins peuvent être couverts par des bibliothèques externes, comme Howler.js pour la gestion audio. Ce choix implique aussi que dans le cas d'une envie

de passer l’affichage en 3D une refonte importante sera nécessaire car PixiJS ne supporte que la 2D.

3.3 Authentication

Afin de proposer une expérience personnalisée, le jeu nécessite la mise en place d’un système d’authentification permettant d’associer des données aux utilisateurs.

Le Framework Play propose des mécanismes d’authentification intégrés. Toutefois, leur utilisation implique de gérer plusieurs aspects sensibles, tels que le stockage sécurisé des mots de passe (hash), la gestion des connexions ainsi que le traitement des erreurs associées.

Une analyse des pratiques adoptées par des applications web populaires, telles que Agar.io, montre que l’authentification déléguée constitue une approche largement répandue. Celle-ci repose généralement sur des protocoles standards comme OAuth2 [9], souvent complétés par OpenID Connect, permettant de déléguer l’authentification à des fournisseurs tiers tels que Google ou GitHub.

Cette approche présente plusieurs avantages. Elle permet de réduire la complexité de développement, d’améliorer la sécurité en s’appuyant sur des acteurs spécialisés, et d’offrir une meilleure expérience utilisateur grâce à des mécanismes de connexion simplifiés.

Ainsi, le recours à une solution d’authentification déléguée permet de concentrer les efforts de développement sur les fonctionnalités principales du jeu, tout en limitant les risques liés à la gestion directe des données sensibles.

3.4 Connection Client-Serveur

Le serveur backend doit établir des connexions avec les utilisateurs afin de pouvoir échanger des messages avec ces derniers. C’est grâce à ces connexions que la communication entre le client et le serveur est assurée. Il doit également assurer le maintien de ces connexions et être capable de les rétablir en cas d’interruption.

3.4.1 Connexions bidirectionnelles

Compte tenu du volume et de la fréquence des échanges, un canal de communication bidirectionnel persistant, tel que les WebSockets, constitue une solution plus adaptée que des approches traditionnelles comme le polling HTTP [10], utilisé avant l'apparition des WebSockets. Cette technique implique des requêtes répétées du client vers le serveur, ce qui engendre une surcharge inutile.

Une autre alternative est l'utilisation des Server-Sent Events (SSE) [11], qui permettent au serveur d'envoyer des mises à jour au client. Toutefois, cette solution reste unidirectionnelle et ne permet pas au client de communiquer directement avec le serveur, ce qui la rend inadaptée dans le cadre d'un jeu interactif.

Le framework Play propose une intégration native des WebSockets [12] pour gérer les communications entre le backend et les utilisateurs. Deux approches principales sont disponibles : une première basée sur Akka Streams et le modèle d'acteurs [13], et une seconde utilisant directement Akka Streams pour manipuler les flux de données.

Le choix s'oriente vers l'approche basée sur le modèle d'acteurs. En effet, ce modèle est particulièrement adapté à la gestion d'états concurrents. Il permet notamment d'associer un acteur à chaque client afin de gérer son état individuel, ainsi qu'un ou plusieurs acteurs supplémentaires pour représenter l'état global de la partie.

3.4.2 Gestions des connexions

Pour assurer le maintien des connexions, il sera nécessaire de mettre en place un système de heartbeat ainsi qu'un mécanisme de reconnexion en cas de déconnexion.

Concernant le système de reconnexion, l'objectif principal est de pouvoir envoyer l'état actuel de la partie au moment où l'utilisateur se reconnecte. Pour cela, il sera nécessaire de conserver un historique de l'état de la partie, cohérent entre tous les utilisateurs. Il s'agit d'un problème classique de concurrence dans un environnement distribué. Dans ce contexte, on pourra s'inspirer de l'algorithme Raft [14] afin d'élaborer notre propre solution.

Cependant, plusieurs problèmes restent à résoudre, notamment la taille de l'historique à conserver, ainsi que la manière dont la reconnexion sera présentée à l'utilisateur. Par exemple, les combats peuvent-ils être rejoués séquentiellement, ou alors est-ce que l'on affiche seulement les résultats, ou bien est-ce que l'utilisateur ne va avoir aucune indication des événements passés ?

3.5 Lien entre DB et Backend

L'application va avoir besoin de stocker des données sur les utilisateurs, que l'on devra créer, éditer, etc. Nous aurons donc besoin de créer des requêtes pertinentes pour accéder à ces données et faire le lien entre notre backend sur Play et notre base de données PostgreSQL. Nous avons à disposition plusieurs solutions pour cela : utiliser du SQL pur, JOOQ [15] ou encore un ORM.

Le premier problème auquel nous serons confrontés est l'impedance mismatch, c'est-à-dire la dissonance entre le modèle orienté objet que nous utilisons et le modèle relationnel de la base de données. Pour résoudre ce problème, la solution la plus simple et la plus économique serait d'utiliser un ORM, dont le but principal est justement de gérer cette différence. De plus, Play propose des tutoriels pour l'intégration d'ORM tels que Hibernate [16], qui est très utilisé dans l'industrie.

Cependant, l'abstraction proposée par les ORM s'avère souvent « leaky », et ces derniers ne constituent pas une solution à tous nos problèmes. En effet, leur utilisation nécessite une attention particulière afin de préciser au maximum nos intentions. Sans cela, l'ORM pourrait générer des structures inefficaces, comme des tables inutiles. De plus, il faut être vigilant quant aux problèmes de performance pouvant survenir dans les requêtes générées.

Néanmoins, les ORM restent une bonne solution pour la plupart des requêtes liées à un utilisateur particulier. Cependant, dans le cas où l'on souhaite implémenter un leaderboard ou effectuer des statistiques sur les personnages, cela nécessitera des requêtes plus complexes. Dans ces situations, écrire des requêtes en SQL pur ou utiliser JOOQ pourrait s'avérer plus approprié.

3.6 CI/CD

Sachant que ce projet a pour objectif de mettre en place un pipeline CI/CD, nous allons nous intéresser aux différentes solutions disponibles.

Tout d'abord, afin de garantir un environnement standardisé pour les différents builds tout au long du projet, nous utiliserons Docker et son système d'images.

Ensuite, après avoir étudié les outils utilisés par différents acteurs du domaine, il apparaît que Jenkins est une solution populaire en raison de sa flexibilité et de son caractère open source. Il permet de ne pas tout construire soi-même grâce à l'utilisation de plugins créés et maintenus par une large communauté. Cependant, son utilisation demande un certain investissement en temps, et nécessiterait la mise en place d'un serveur dédié pour exécuter Jenkins. De plus, les retours de développeurs ayant travaillé avec cet outil indiquent que la maintenance des pipelines peut devenir chronophage, notamment en raison de problèmes de compatibilité entre certains plugins.

Une alternative plus simple à mettre en œuvre serait d'utiliser GitHub Actions. Cette solution est particulièrement pratique, puisque le versionnement du projet sera déjà réalisé sur GitHub. Elle permet une intégration rapide et ne nécessite pas de serveur supplémentaire pour exécuter les pipelines, sauf en cas d'utilisation de runners auto-hébergés.

Finalement, bien qu'il existe des tutoriels réalisés par Riot Games (les développeurs de TFT) montrant une intégration basée sur Jenkins et Docker [17], nous choisirons d'utiliser GitHub Actions afin de gagner du temps et de simplifier la mise en place du pipeline. De plus, il n'y aura qu'une partie Continuous Delivery dans le premier temps où la création et la publication de l'image docker sera automatiser.

4 Architecture Logiciel

Après avoir analysé les différentes solutions qui nous étaient à disposition on va maintenant vous présenter l'architecture logiciel choisie ainsi qu'une brève présentation de ces technologies.

4.1 Frontend

Le frontend soit l'interface client va être développé en JavaScript notamment avec l'aide de la librairie PixieJS

4.1.1 PixiJS

PixiJS est un moteur de rendu 2D open source permettant de créer des visuels de haute qualité sur le web à l'aide de JavaScript. Ce moteur est construit sur WebGL et, de manière optionnelle, sur WebGPU, ce qui lui permet d'offrir des performances élevées et d'exécuter sans difficulté des scènes contenant de nombreux sprites.

Dans le cadre de mon projet, PixiJS gère l'affichage de nombreux Sprites comme tous les personnages ou encore l'interface utilisateur, mais aussi toutes les animations de combats. Il permet ainsi de supporter un grand nombre d'éléments graphiques simultanés sans dégradation significative des performances.

4.2 Backend

Le backend va être en Java, il va exécuter toute la logique métier de l'application donc, faire des authentifications, gérer les connexions avec les utilisateurs, gérer les parties, simuler les combats, etc...

4.2.1 Play

Le Play Framework est un framework web open source conçu pour le développement d'applications web en Java et Scala. Il suit une architecture moderne basée sur le modèle MVC (Model-View-Controller), ce qui permet de structurer clairement le code et de faciliter la maintenance des applications.

Play se distingue par son approche réactive et non bloquante, ce qui lui permet de gérer efficacement un grand nombre de requêtes simultanées. Il est également fortement intégré à l'écosystème Java, ce qui le rend compatible avec de nombreuses bibliothèques et outils utilisés en entreprise.

Dans le cadre de ce projet, Play Framework est utilisé pour implémenter la partie backend de l'application, notamment la gestion des connexions avec le pattern acteur et pekko stream, tout cela sans soucis de concurrence.

4.2.2 OAuth2/OpenID connect

OAuth 2.0 est un protocole d'autorisation largement utilisé qui permet à une application d'accéder à des ressources protégées au nom d'un utilisateur, sans avoir besoin de connaître ses identifiants. Il repose sur l'émission de tokens d'accès, qui définissent des permissions limitées et temporaires.

OpenID Connect est une couche d'authentification construite au-dessus d'OAuth 2.0. Elle permet de vérifier l'identité d'un utilisateur et de récupérer des informations de profil de manière standardisée, via un mécanisme basé sur des tokens d'identité (ID Token).

Dans le cadre de ce projet, OAuth 2.0 et OpenID Connect sont utilisés pour sécuriser l'authentification des utilisateurs et centraliser la gestion des accès en faisant appel à un gestionnaire tiers (google, github). Cela, permet de ne pas se préoccuper de toute la partie sécurité, stockage d'empreinte de mot de passe et identification.

4.2.3 Hibernate

Hibernate est un framework de type ORM (Object-Relational Mapping) utilisé en Java pour faciliter la persistance des objets en base de données relationnelle. Il permet de faire le lien entre le modèle objet de l'application et le modèle relationnel de la base de données, en automatisant la génération des requêtes SQL.

Grâce à Hibernate, les développeurs peuvent manipuler directement des objets Java sans avoir à écrire explicitement la majorité des requêtes SQL. Le framework se charge de la traduction entre les objets et les tables, ainsi que de la gestion des relations, des transactions et du cycle de vie des entités.

Dans le cadre de ce projet, Hibernate est utilisé pour simplifier l'accès à la base de données et réduire la complexité liée à l'écriture et à la maintenance des requêtes SQL.

4.3 Base de données

4.3.1 PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle (SGBDR) open source, reconnu pour sa robustesse, sa conformité aux standards SQL et ses performances. Il permet de stocker, organiser et interroger des données structurées à l'aide du langage SQL.

PostgreSQL offre de nombreuses fonctionnalités avancées, telles que la gestion des transactions ACID, le support des index performants, ainsi que la possibilité de définir des types de données personnalisés et des requêtes complexes. Il est également fortement apprécié pour sa fiabilité et sa capacité à gérer de grandes quantités de données.

Dans le cadre de ce projet, PostgreSQL est utilisé comme base de données principale afin de stocker les informations liées aux utilisateurs et aux données applicatives, tout en garantissant la cohérence et l'intégrité des données.

4.4 CI/CD Pipeline

Le but de notre CI/CD pipeline serait d'automatiser la parti build et test de chaque nouvel release tout en étant sur que il n'y aura pas de surprise car les tests auront été réalisé sur le même environnement que celui qui sera utiliser pour le serveur. De plus, il n'y aura aussi que de la continuous delivery, et donc pas de deployment automatique.

4.4.1 Github actions

GitHub Actions est une plateforme d'intégration et de déploiement continu (CI/CD) intégrée directement à GitHub. Elle permet d'automatiser des workflows tels que les tests, la compilation, le linting ou encore le déploiement d'une application à chaque modification du code source.

Les workflows GitHub Actions sont définis sous forme de fichiers YAML et s'exécutent automatiquement en réponse à des événements du dépôt, comme un push, une pull request ou la création d'une release. Cette approche permet de standardiser et fiabiliser les processus de développement.

Dans le cadre de ce projet, GitHub Actions est utilisé pour automatiser les tests et les étapes de déploiement, afin de garantir la qualité du code tout en simplifiant la mise en place du pipeline CI/CD.

4.4.2 Docker

Docker est une plateforme open source permettant de créer, déployer et exécuter des applications dans des conteneurs. Un conteneur est un environnement isolé qui regroupe une

application ainsi que toutes ses dépendances, garantissant ainsi un comportement identique quel que soit le système hôte.

Grâce à cette approche, Docker simplifie la gestion des environnements de développement, de test et de production, en réduisant les problèmes liés aux différences de configuration entre machines. Les conteneurs sont légers, rapides à démarrer et plus efficaces en ressources que les machines virtuelles traditionnelles.

Dans le cadre de ce projet, Docker est utilisé afin de standardiser les environnements d'exécution et de faciliter le déploiement des différentes parties de l'application, notamment le backend et les services associés.

4.5 Schéma de l'architecture

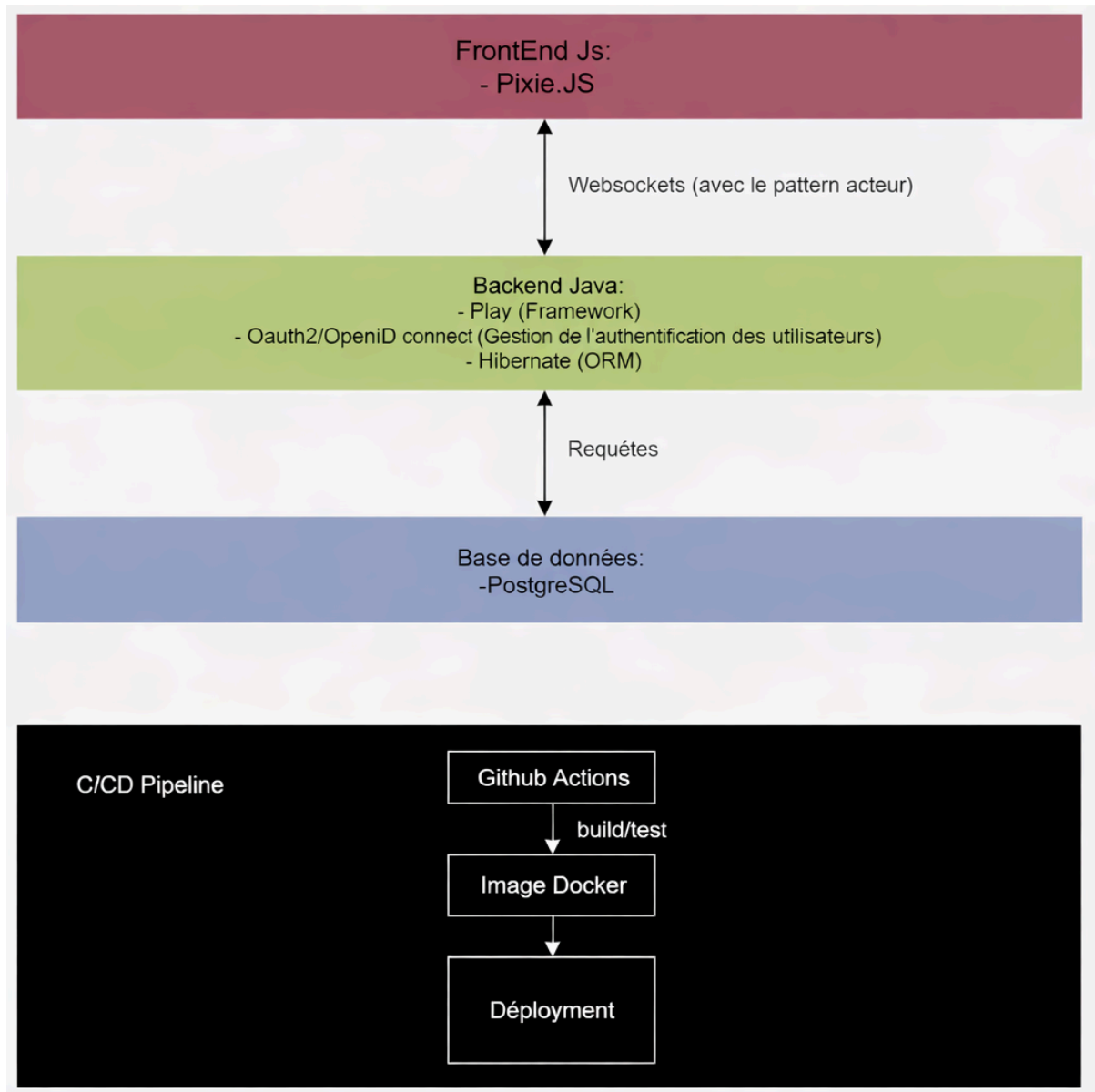


Fig. 2. – Schéma architecture logiciel

Schéma

5 Conception

Pour la partie conception, on va d'abord se concentrer sur la partie base de données pour mettre au clair toutes les données que j'ai besoin de stocker ainsi que les données que je veux stocker. Pour l'instant, on va mettre de côté l'objectif de faire un système de replay des parties, que l'on conceptualisera une fois le nécessaire conceptualisé.

Disclaimer, ici beaucoup de choses sont dépendantes de mon jeu (comment il marche, ce qu'il est intéressant de stocker ou non), je vous invite donc à regarder le Game Design Document (GDD) présent dans l'annexe pour avoir une meilleure compréhension du jeu.

5.1 Conception de la base de données

5.1.1 Les données statiques

Tout d'abord, pour qu'une partie se déroule sans soucis, il nous faut des données (unités, objets, etc..), et afin que le jeu puisse perdurer dans le temps et que je puisse ajouter ou retirer des unités, objets ou autres, je vais stocker ces données « static » sur ma base de données afin que je n'aie pas besoin d'accéder au code pour ça.

On va donc s'intéresser aux données essentielles dont mon jeu a besoin pour faire fonctionner une partie. Pour cela, il m'est nécessaire de stocker :

- Les unités qui peuvent être jouées : leur nom, stats, sprite, etc..
- Les objets que les unités peuvent avoir : leur nom, sprite, etc..
- Les traits que les unités sont censées avoir ainsi que les bonus venant de ces traits
- Les effets qui décrivent les différents bonus et malus des traits, des objets et des unités.

Enfin, comme les unités ont un certain pourcentage de chance d'apparaître en fonction du niveau des joueurs, il faut que je stocke ça. De plus, comme le jeu peut être voué à changer et à être équilibré, chaque catégorie individuelle aura le patch auquel elle appartient. Ça nous donne ceci :

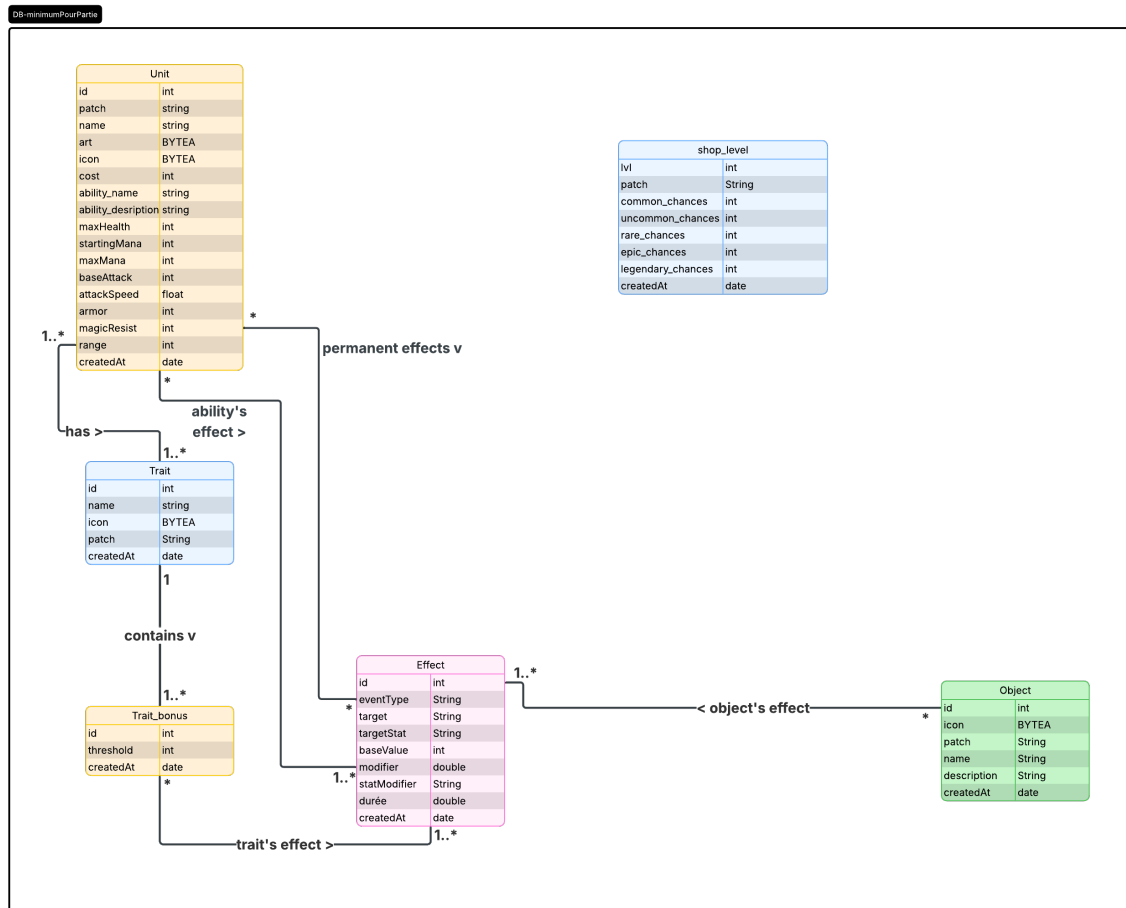


Fig. 3. – Schéma EA Unité

L'association permanent effect fera plus de sens plus tard, mais comme j'ai pour objectif de pouvoir ajouter des modificateurs permanents et que donc la source de cet effet ne fait peut-être plus partie de ton équipe, alors je dois le stocker.

5.1.2 La résistance aux pannes

Maintenant qu'on a les données "static", qui ne bougeront qu'en cas de futurs patches pour équilibrer le jeu. J'ai maintenant un souci dans le cas où un utilisateur perd sa connexion à mon serveur, je dois pouvoir lui renvoyer son état tel qu'il était avant sa déconnexion.

Je pourrais donc stocker cette information sur le serveur, mais si mon serveur tombe en panne ou s'éteint entre-temps, je ne pourrais plus lui redonner cette information. Il faut donc que je la stocke dans ma base de données. Et au passage, comme je dois implémenter un système de replay, je vais regarder les deux.

Car je vais considérer les parties comme un enchaînement d'événements, j'aurai alors juste besoin de rejouer les événements pour faire mon replay. Et pour la tolérance aux pannes, c'est pareil. Donc l'objectif est de stocker les actions des joueurs, ici les événements.

5.1.2.1 Event Sourcing vs Relationnel

Pour ça, j'ai deux solutions : la première vers laquelle je suis parti, est d'utiliser le pattern d'événement sourcing, et la deuxième solution est une base de données totalement relationnelle.

Le pattern d'événement sourcing correspond totalement à notre façon de voir les parties [18], et si je l'utilise, cela me permettrait d'avoir un stockage et une architecture de ma DB, et le replay assez simplement. Le but ici serait de stocker tous mes événements et leurs données dans un JSON et de faire des screenshots de la fin des rounds, aussi stockés dans un JSON, pour permettre d'avoir l'état actuel d'une équipe plus rapidement.

Un des plus gros problèmes que l'on a avec cette direction est que l'on perd la cohérence et l'intégrité des données fournies par les BD relationnelles. Il faudrait donc gérer cela dans le backend et c'est donc une chose en plus, sachant qu'il faudra aussi parser tous les JSON stockés si l'on veut les utiliser une fois récupérés depuis la base de données.

Alors que le relationnel, lui, va nous rendre l'architecture de notre BD plus complexe, mais une fois la conception faite, on n'aura plus rien à faire de plus côté backend et notre système de replay et de tolérance aux pannes seront opérationnels.

Je vais donc me tourner vers une BD entièrement relationnelle pour ce projet.

5.1.3 La tolérance aux pannes sans le replay

Maintenant que je sais que je vais faire ça en relationnel, il faut que je regarde comment conceptualiser la tolérance aux pannes et le système de replay.

5.1.3.1 Stockage des équipes

Pour ça, il me faudra :

- Stocker des informations des joueurs : pour qu'ils puissent se connecter et que je puisse les identifier
- Les équipes utilisées par les joueurs :
- leurs unités et leurs objets
- l'inventaire des objets que le joueur a
- les données actuelles de l'équipe (gold, lvl, pv, winstreak, placement, le shop actuel)

J'ai donc toutes les données de l'équipe d'un joueur et je vais aussi stocker les anciens combats afin qu'un joueur puisse voir les derniers combats ainsi que le résultat et qui il a affronté.

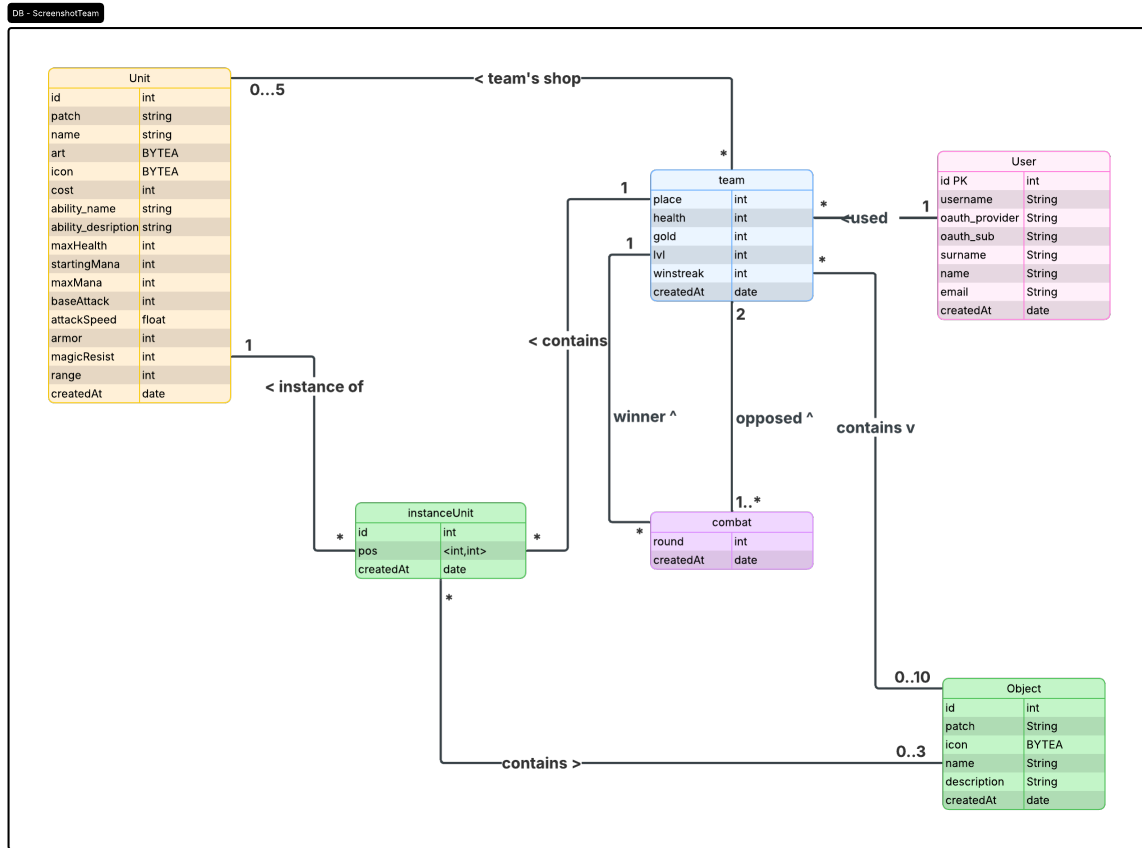


Fig. 4. – Schéma EA Team

5.1.3.2 Stockage des parties

Pour pouvoir récupérer d'une panne, il faut aussi que je sache à quelle partie l'équipe participait et il me faut les données de la partie, soit la partie, sa seed et le pool de champions actuel de la partie. Pour rendre les choses plus simples plus tard, les pools de champions sont séparés en fonction de la rareté des champions à l'intérieur de ces derniers.

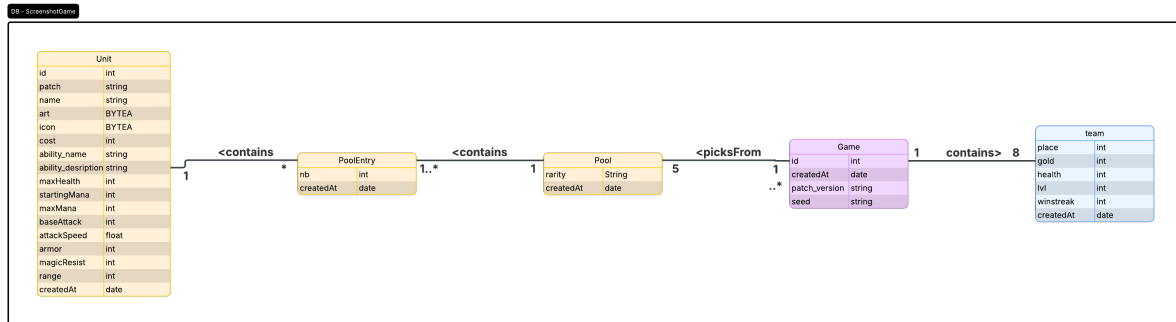


Fig. 5. – Schéma EA Game

5.1.4 Tolérance aux pannes avec le replay

J'ai conceptualisé une BD qui est tolérante aux pannes en stockant l'état actuel du début de round, mais je perds toute action du round actuel lors de la panne. Voici le schéma complet jusqu'à présent.

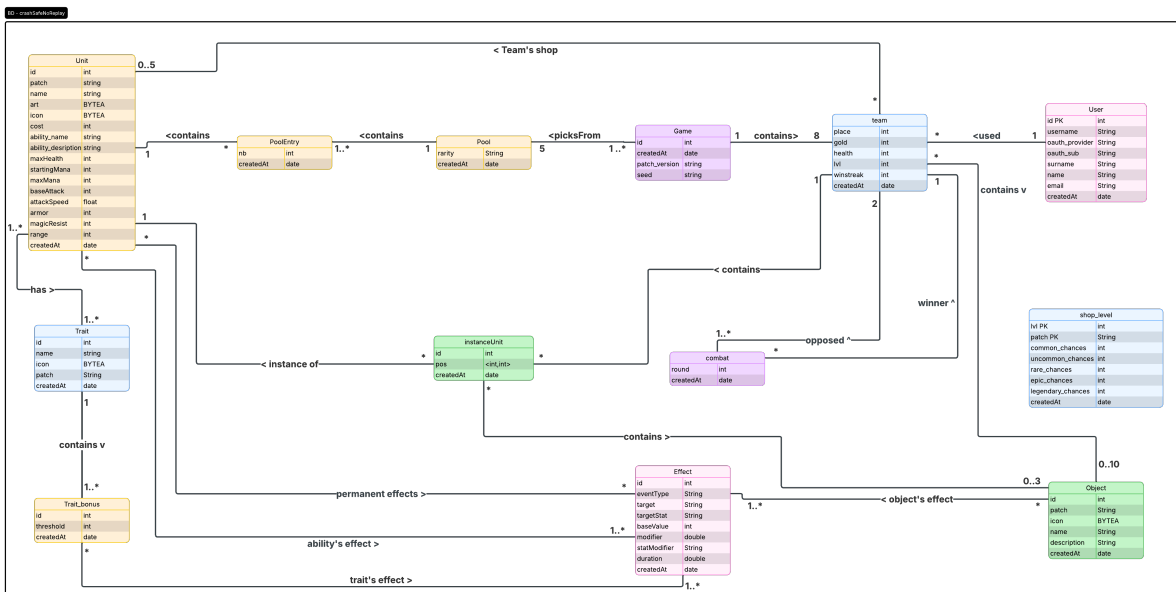


Fig. 6. – Schéma EA SansReplay

5.1.4.1 Description des actions utilisateurs

Maintenant que l'on a tout ça, il faut regarder le principe de replay des parties. Un autre gros avantage de ce replay, c'est que ça permet aux joueurs de se rendre compte de leurs erreurs et potentiellement de les corriger dans le futur.

Cependant, pour cela, il faut comprendre un peu mieux comment se déroule une partie. Dans mon jeu, une partie va consister en plusieurs rounds qui se suivent. Chaque joueur va commencer la partie avec le même niveau d'exp et de gold ainsi qu'une unité de la rareté la plus basse aléatoire. Ensuite, il commence le premier round. Un round est composé de deux phases, chacune durant laquelle on peut faire des actions. Il y a la buying phase et lorsque le temps est écoulé, le joueur bascule dans la fighting phase où il doit affronter un adversaire au hasard.

Voici un diagramme de séquence pour mieux comprendre les actions que l'on a le droit d'effectuer ainsi que ce que cela engendre :

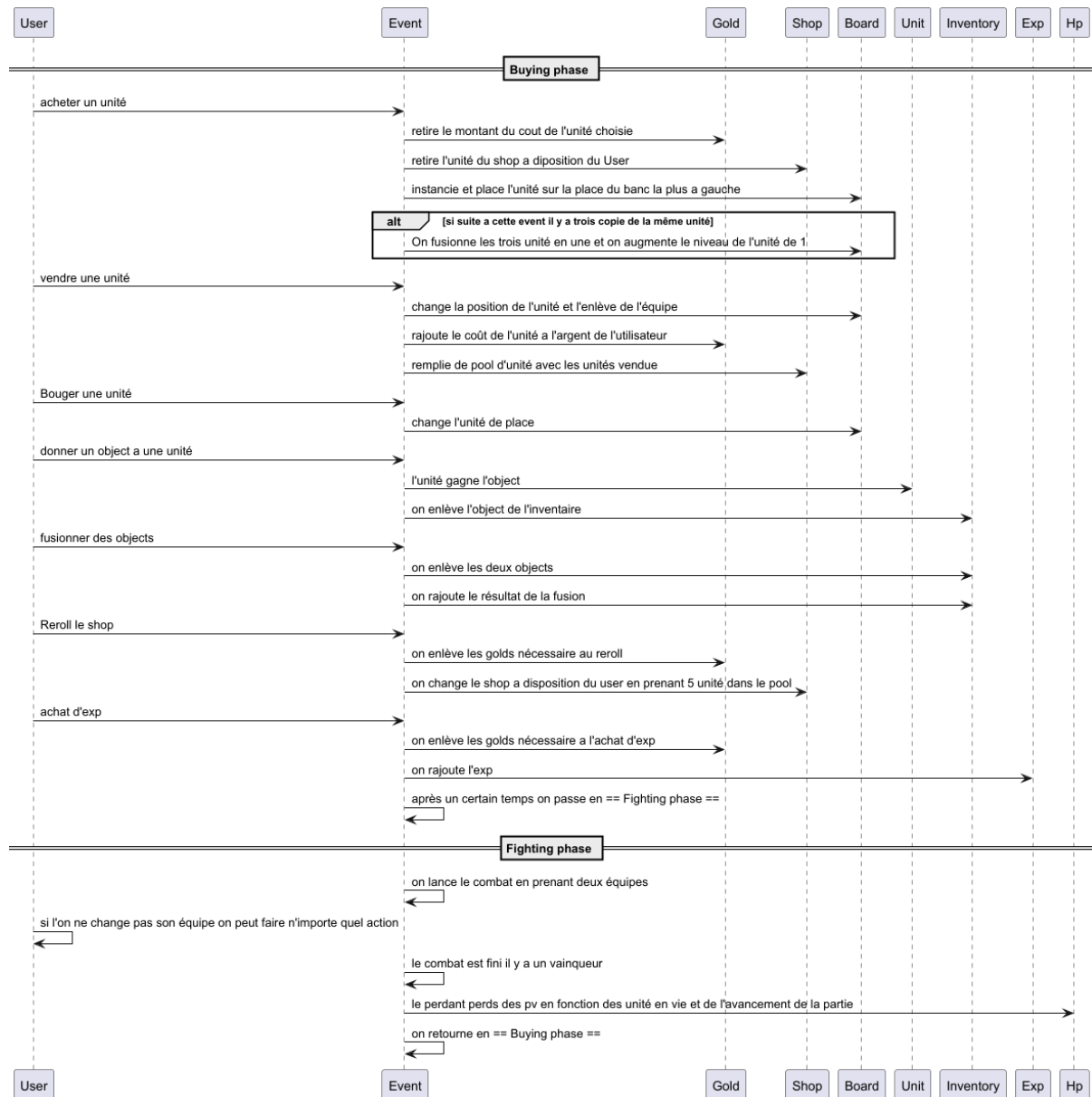


Fig. 7. – Diagramme de séquence des actions pendant un round

5.1.4.2 Conception dans le schéma

Maintenant que nous comprenons mieux les actions des utilisateurs, il paraît évident qu'un système de stockage basé sur les événements réalisés par les utilisateurs est la voie à suivre. Pour cela, nous ne conserverons que les données essentielles à l'événement et qu'il n'est pas possible de retrouver autrement.

Pour donner un exemple, lorsqu'on achète une unité, il faut payer le coût de cette unité. Cependant, il faudra déjà stocker dans la base de données quelle unité a été achetée ; il sera donc possible de retrouver son coût sans aucun problème lors de l'exécution de l'événement. Je n'ai donc pas besoin de stocker également le prix de l'unité dans l'événement.

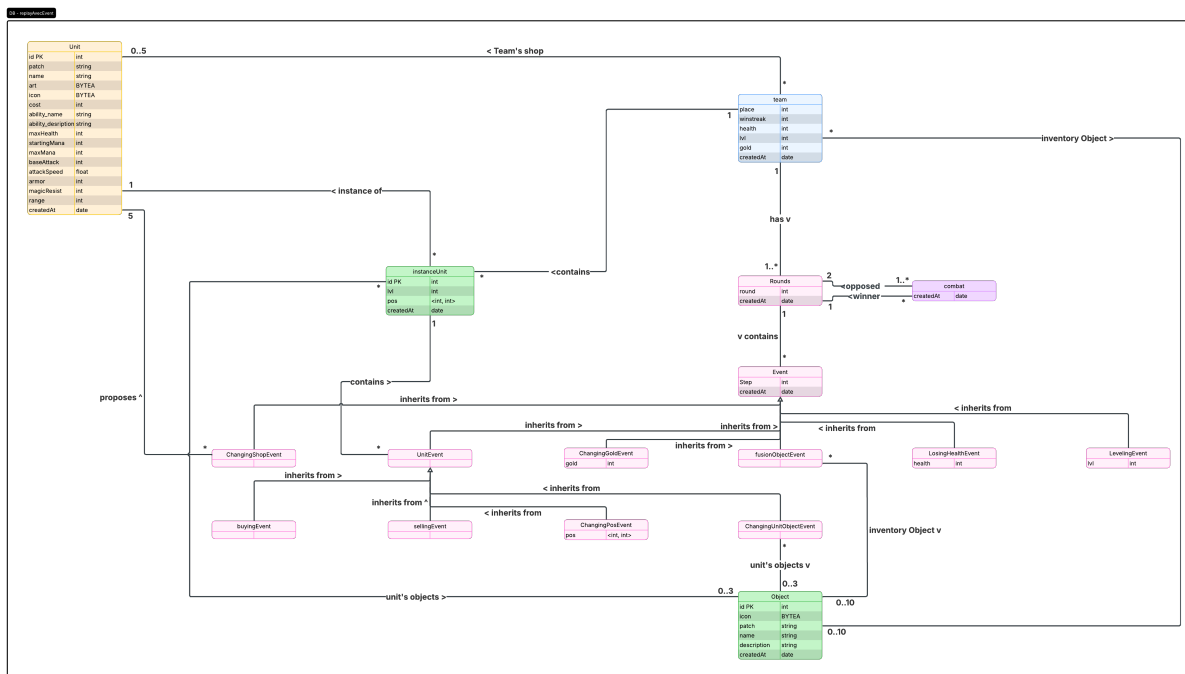


Fig. 8. – Schéma EA Event pour replay

5.1.5 Schéma final de la base de donnée

Maintenant que nous disposons d'une base de données offrant une tolérance aux pannes ainsi qu'un système de replay, tout en stockant les données « statiques » nécessaires au fonctionnement du jeu, nous obtenons le schéma suivant.

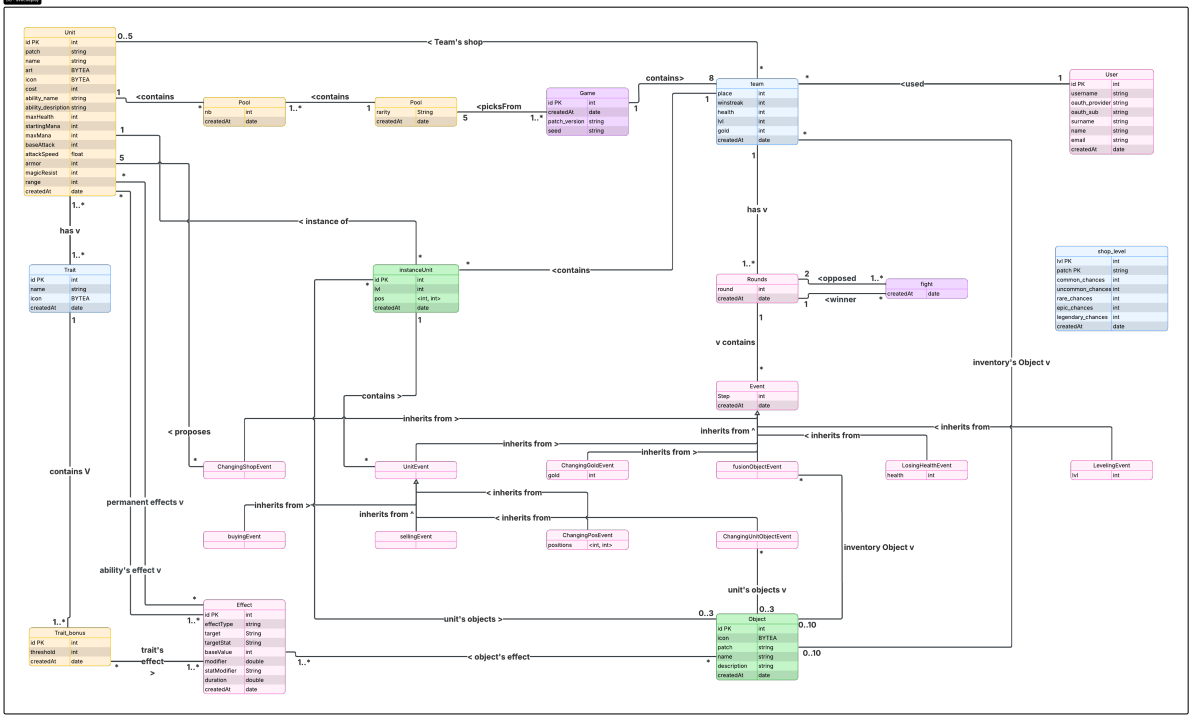


Fig. 9. – Schéma EA

6 Conclusion

Malgré un démarrage un peu lent, le projet a désormais bien avancé, et la finalisation de la conception de la base de données constitue une étape importante dans son développement. Cependant, par rapport à la planification initiale, un retard d'environ deux semaines est à constater. Il sera donc nécessaire d'adapter le rythme de travail afin de rattraper ce décalage. En effet, selon le planning initial, l'implémentation des fonctionnalités *must have* aurait déjà dû être bien entamée, alors que celle-ci ne fait actuellement que commencer.

Les prochaines étapes du projet consisteront donc à implémenter le backend ainsi que le frontend afin d'obtenir une première version fonctionnelle de l'application, sur laquelle les différentes fonctionnalités prévues pourront ensuite être ajoutées progressivement.

Bibliographie

- [1] Play Framework, « Play Framework Documentation 3.0.x ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/Home>
- [2] Play Framework, « Play Framework Scala Templates ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/ScalaTemplates>
- [3] Wikipédia, « WebGL Wikipedia ». [En ligne]. Disponible sur: <https://fr.wikipedia.org/wiki/WebGL>
- [4] PixiJS Team, « PixiJS Official Website ». [En ligne]. Disponible sur: <https://pixijs.com/>
- [5] Phaser Studio, « Phaser Official Website ». [En ligne]. Disponible sur: <https://phaser.io/>
- [6] Reddit Community, « Is It Worth Developing a Game Made Only in HTML/JS? ». [En ligne]. Disponible sur: https://www.reddit.com/r/gamedev/comments/vdp8h0/is_it_worth_developing_a_game_made_only_in/?tl=fr
- [7] Shirajuki, « JS Game Rendering Benchmark ». [En ligne]. Disponible sur: <https://github.com/Shirajuki/js-game-rendering-benchmark>
- [8] slaylines, « Canvas Engines Comparison ». [En ligne]. Disponible sur: <https://github.com/slaylines/canvas-engines-comparison>
- [9] Aaron Parecki, « Oauth2 ». [En ligne]. Disponible sur: <https://oauth.net/2/>
- [10] Ably, « What is Long Polling? ». [En ligne]. Disponible sur: <https://ably.com/topic/long-polling>
- [11] MDN Web Docs, « Using Server-Sent Events ». [En ligne]. Disponible sur: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- [12] Play Framework, « Play Framework Java WebSockets ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/JavaWebSockets>
- [13] Wikipédia, « Modèle d'acteur ». [En ligne]. Disponible sur: https://fr.wikipedia.org/wiki/Mod%C3%A8le_d%27acteur
- [14] Olivier Lemer, « Raft ». [En ligne]. Disponible sur: <https://sdr-classroom.github.io/slides/7-1-consensus-raft/>
- [15] Lukas Eder, « Using jOOQ with Play Framework ». [En ligne]. Disponible sur: <https://blog.jooq.org/using-jooq-with-play-framework/>

- [16] Play Framework, « Play Framework Database Documentation ». [En ligne]. Disponible sur: <https://www.playframework.com/documentation/3.0.x/Databases>
- [17] Riot Games, « Thinking Inside Container ». [En ligne]. Disponible sur: <https://www.riotgames.com/en/news/thinking-inside-container>
- [18] GenesisDB, « What is event sourcing? ». [En ligne]. Disponible sur: <https://www.eventsourcing.dev/what-is-event-sourcing>

Table des figures

Fig. 1	Plannification	13
Fig. 2	Schéma architecture logiciel	25
Fig. 3	Schéma EA Unité	27
Fig. 4	Schéma EA Team	30
Fig. 5	Schéma EA Game	31
Fig. 6	Schéma EA SansReplay	31
Fig. 7	Diagramme de séquence des actions pendant un round	33
Fig. 8	Schéma EA Event pour replay	34
Fig. 9	Schéma EA	35

Annexes

I Outils utilisés

Durant cce project voici mon utilisation des LLMs:

- Correction de l'orthographe
- Quelques propositions de reformulation de phrase
- Explication de sujet théorique
- Aide à la recherche de source sur les Technologies utilisées dans mes inspirations (TFT, hearthstone)